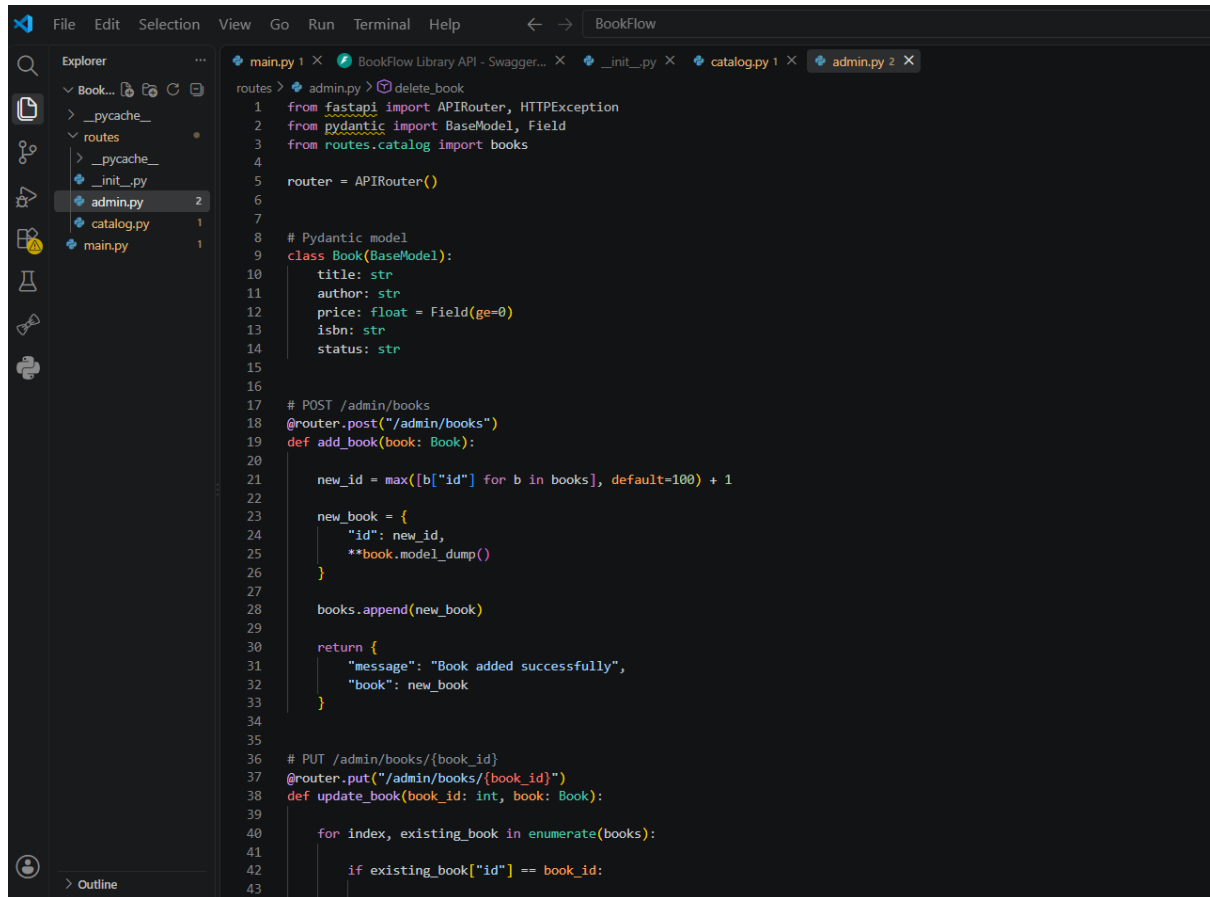


PRACTICAL WEEK 6

2520090060

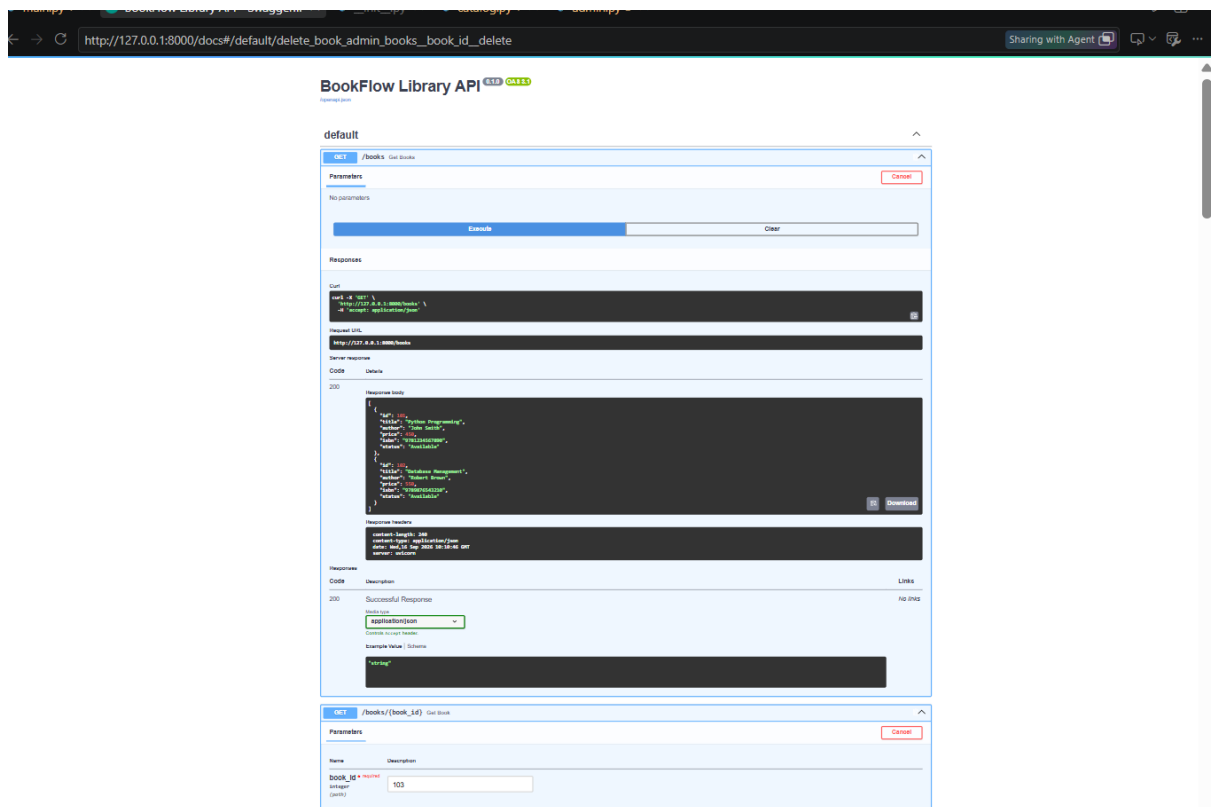
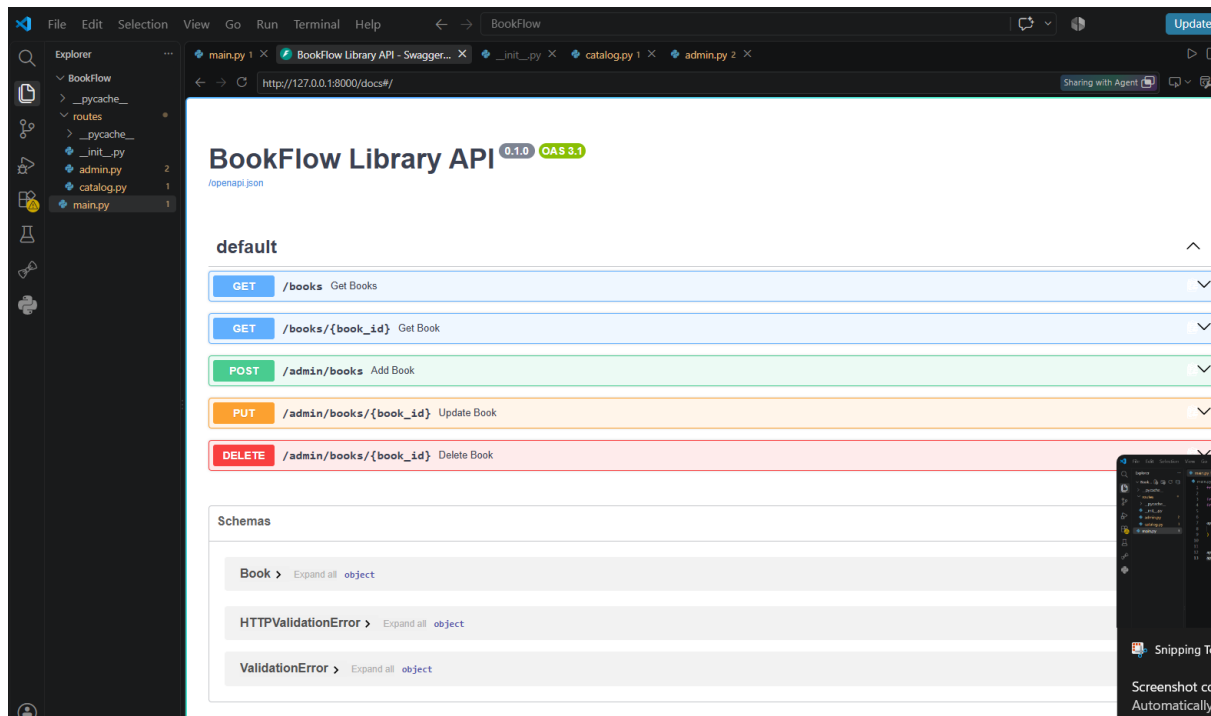
P.VEDADRI NAIDU



```
1 from fastapi import APIRouter, HTTPException
2 from pydantic import BaseModel, Field
3 from routes.catalog import books
4
5 router = APIRouter()
6
7
8 # Pydantic model
9 class Book(BaseModel):
10     title: str
11     author: str
12     price: float = Field(ge=0)
13     isbn: str
14     status: str
15
16
17 # POST /admin/books
18 @router.post("/admin/books")
19 def add_book(book: Book):
20
21     new_id = max([b["id"] for b in books], default=100) + 1
22
23     new_book = {
24         "id": new_id,
25         **book.model_dump()
26     }
27
28     books.append(new_book)
29
30     return {
31         "message": "Book added successfully",
32         "book": new_book
33     }
34
35
36 # PUT /admin/books/{book_id}
37 @router.put("/admin/books/{book_id}")
38 def update_book(book_id: int, book: Book):
39
40     for index, existing_book in enumerate(books):
41
42         if existing_book["id"] == book_id:
```

```
1 from fastapi import APIRouter, HTTPException
2
3 router = APIRouter()
4
5 # Shared book data
6 books = [
7     {
8         "id": 101,
9         "title": "Python Programming",
10        "author": "John Smith",
11        "price": 450,
12        "isbn": "9781234567890",
13        "status": "Available"
14    },
15    {
16        "id": 102,
17        "title": "Database Management",
18        "author": "Robert Brown",
19        "price": 550,
20        "isbn": "9789876543210",
21        "status": "Available"
22    }
23 ]
24
25
26 # GET /books
27 @router.get("/books")
28 def get_books():
29     return books
30
31
32 # GET /books/{book_id}
33 @router.get("/books/{book_id}")
34 def get_book(book_id: int):
35
36     for book in books:
37         if book["id"] == book_id:
38             return book
39
40     raise HTTPException(
41         status_code=404,
42         detail="Book not found"
43     )
```

```
1 from fastapi import FastAPI
2
3 from routes.catalog import router as catalog_router
4 from routes.admin import router as admin_router
5
6
7 app = FastAPI(
8     title="BookFlow Library API"
9 )
10
11
12 app.include_router(catalog_router)
13 app.include_router(admin_router)
```



GET /books/{book_id} Get Book

Parameters

Name	Description
book_id	required integer (path)

Execute Clear

Responses

Curl

```
curl -X GET \
  http://127.0.0.1:3000/books/103 \
  -H 'accept: application/json'
```

Request URL

http://127.0.0.1:3000/books/103

Server response

Code Details

404

Error: Not Found

Response body

```
{
  "detail": "Book not found"
}
```

Response headers

```
content-length: 27
content-type: application/json
date: Wed, 16 Sep 2015 16:16:17 GMT
server: nodejs
```

Responses

Code	Description	Links
200	Successful Response	No links

Media type: application/json

Example Value | Schema

```
"string"
```

422

Validation Error

Media type: application/json

Example Value | Schema

```
{
  "detail": {
    "type": {
      "enum": [
        "string"
      ]
    },
    "value": "string"
  }
}
```

POST /admin/books Add Book

Parameters

No parameters

Request body

Media type: application/json

Body Value | Schema

```
{
  "title": "Artificial Intelligence",
  "author": "John Doe",
  "price": 700,
  "isbn": "9781111111111",
  "status": "Available"
}
```

Execute Clear

Responses

Curl

```
curl -X POST \
  http://127.0.0.1:3000/admin/books \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "title": "Artificial Intelligence",
    "author": "John Doe",
    "price": 700,
    "isbn": "9781111111111",
    "status": "Available"
  }'
```

Request URL

http://127.0.0.1:3000/admin/books

Server response

Code Details

200

Response body

```
{
  "message": "Book added successfully",
  "book": {
    "title": "Artificial Intelligence",
    "author": "John Doe",
    "price": 700,
    "isbn": "9781111111111",
    "status": "Available"
  }
}
```

Response headers

```
content-length: 169
content-type: application/json
date: Wed, 16 Sep 2015 16:11:56 GMT
server: nodejs
```

Responses

Code	Description	Links
200	Successful Response	No links

Media type: application/json

Example Value | Schema

```
"string"
```

422

Validation Error

Media type: application/json

DELETE /admin/books/{book_id}
Delete Book

Parameter:
Cancel

Name
Description

book_id *REQUIRED
103

Integer
(path)

Execute
Clear

Responses

Curl

curl -X DELETE \
 'http://127.0.0.1:8080/admin/books/103' \
 -H 'accept: application/json'

Rawtext URL

http://127.0.0.1:8080/admin/books/103

Server response

Code
Details

200

Response body

{
 "message": "Book deleted successfully",
 "meta": {
 "id": 103,
 "title": "Advanced Artificial Intelligence",
 "author": "Steve Nouri",
 "year": 2019,
 "status": "AVAILABLE",
 "status": "Available"
 }
}

Download

Response headers

content-length: 185
content-type: application/json
date: Wed, 14 Sep 2022 16:12:31 GMT
server: nginx

Responses

Code
Description

200

200

Response body

```
{
  "message": "Book deleted successfully",
  "user": {
    "id": 1,
    "title": "Advanced Artificial Intelligence",
    "author": "John Doe",
    "year": 2022,
    "isbn": "9781111111111",
    "status": "available"
  }
}
```

Download

Response headers

```
content-length: 186
content-type: application/json
date: Sat, 10 Sep 2022 20:11:23 GMT
server: nginx
```

Responses

Code	Description	Links
200	Successful Response	API docs
Media type: application/json Example Value Schema <pre>"wrong"</pre>		
422	Validation Error	API docs
Media type: application/json Example Value Schema <pre>{ "detail": { "type": { "name": "wrong", "type": "string", "value": "wrong" }, "code": "wrong" } }</pre>		

Schemas

Book > Expand all object

HTTPValidationError > Expand all object

ValidationError > Expand all object